NoSQL und KI-Datenbanksysteme InfluxDB

1 Einrichtung InfluxDB (Setup)

Image herunterladen

Lade das offizielle InfluxDB-Image aus dem Docker Hub:

```
docker image pull influxdb:latest
```

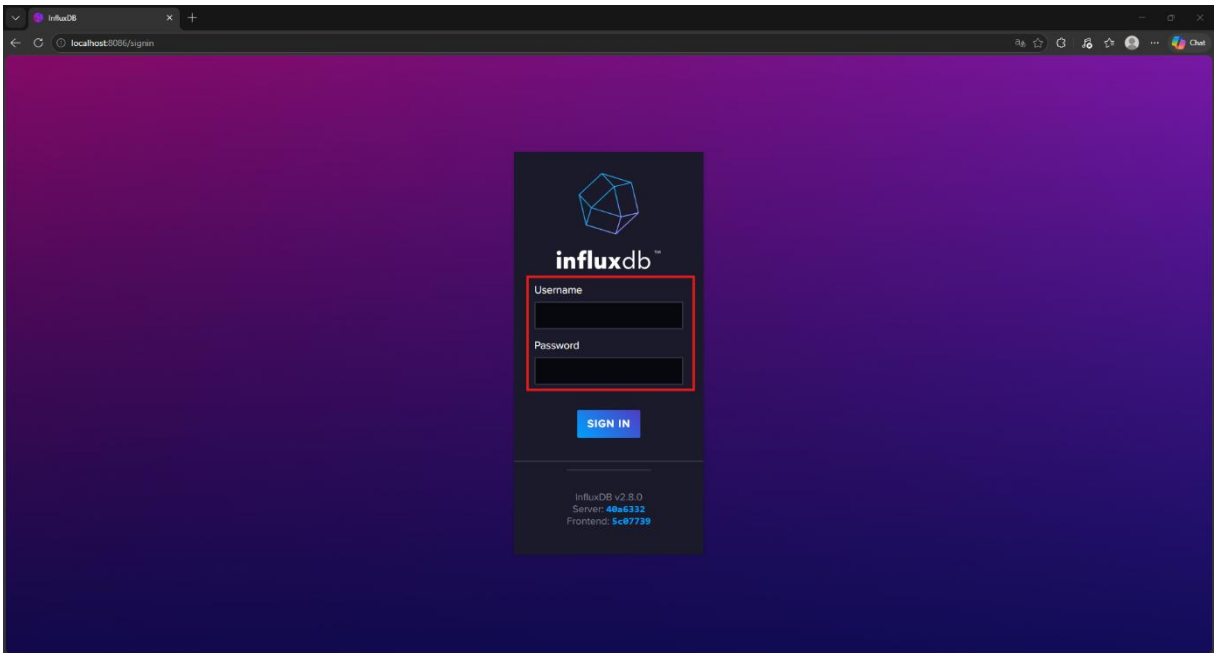
Container erstellen und konfigurieren

Erzeuge den Container mit den notwendigen Umgebungsvariablen für die Erstkonfiguration:

```
docker run -d `
  --name influxdb `
  -p 8086:8086 `
  -v $HOME/influxdb/data:/var/lib/influxdb2 `
  -e DOCKER_INFLUXDB_INIT_MODE=setup `
  -e DOCKER_INFLUXDB_INIT_USERNAME=admin `
  -e DOCKER_INFLUXDB_INIT_PASSWORD=password123 `
  -e DOCKER_INFLUXDB_INIT_ORG=my-lab-org `
  -e DOCKER_INFLUXDB_INIT_BUCKET=lab-bucket `
  influxdb:latest
```

Zugriff (Dashboard)

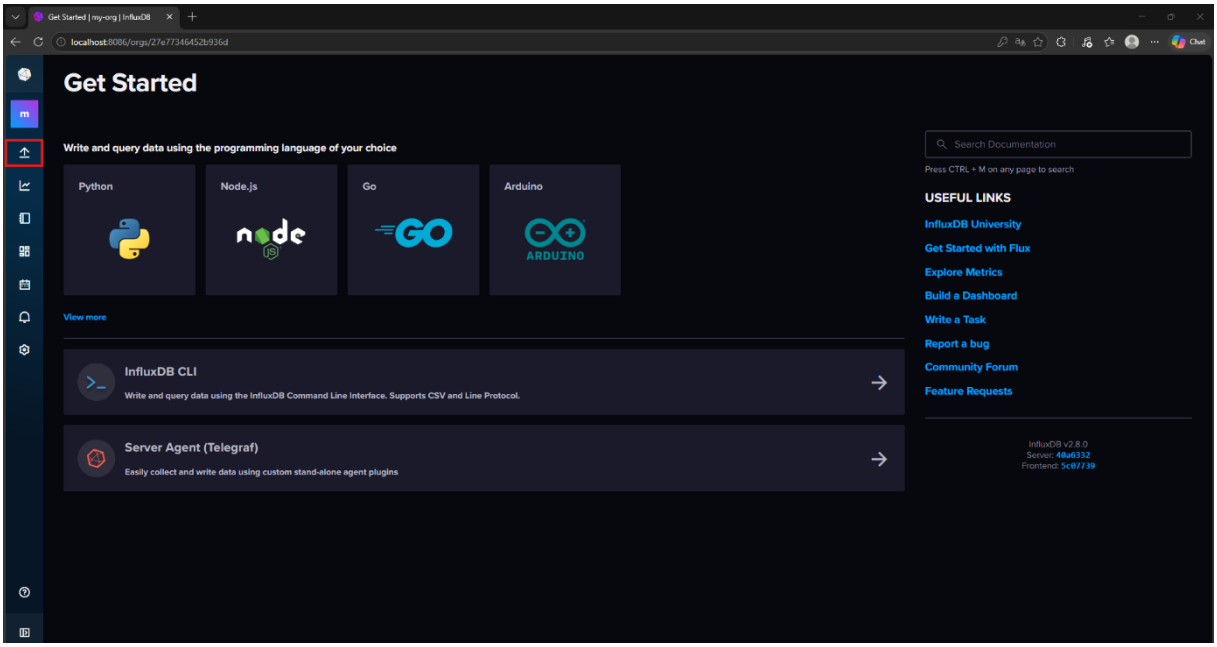
Öffne den Browser unter **http://localhost:8086**. Logge dich mit den oben gewählten Daten **(admin / password123)** ein.



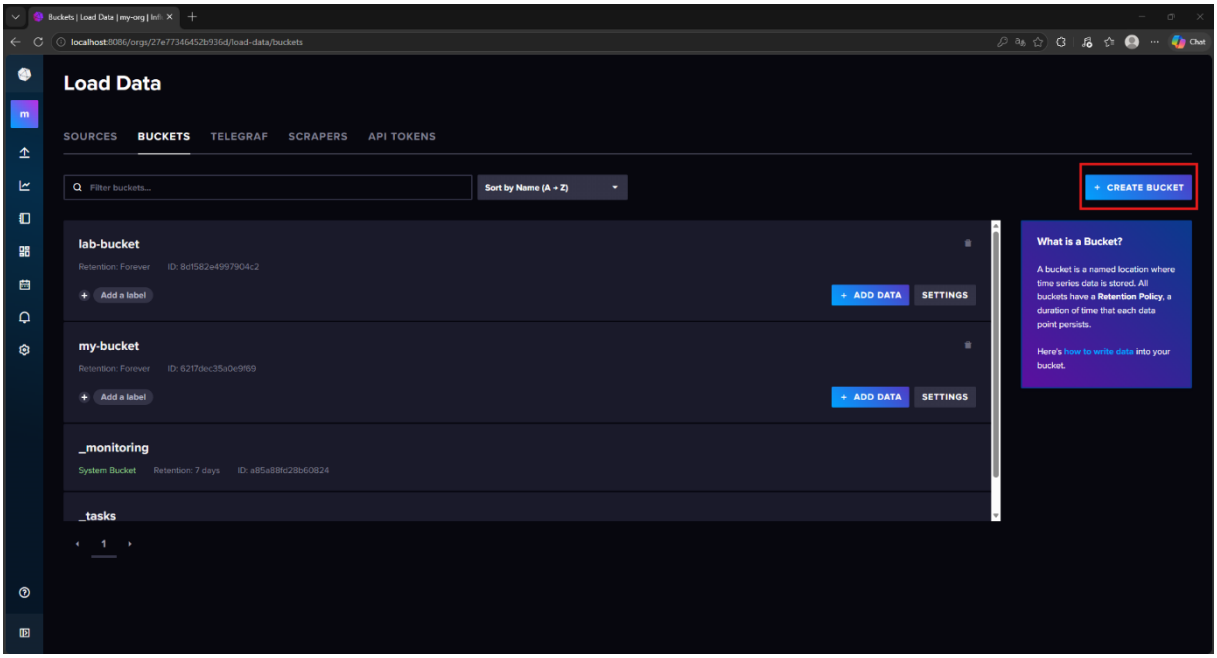
2. Datenimport (Line Protocol)

In einer Relationalen Datenbanken nutzt man CREATE, in InfluxDB das Line Protocol.

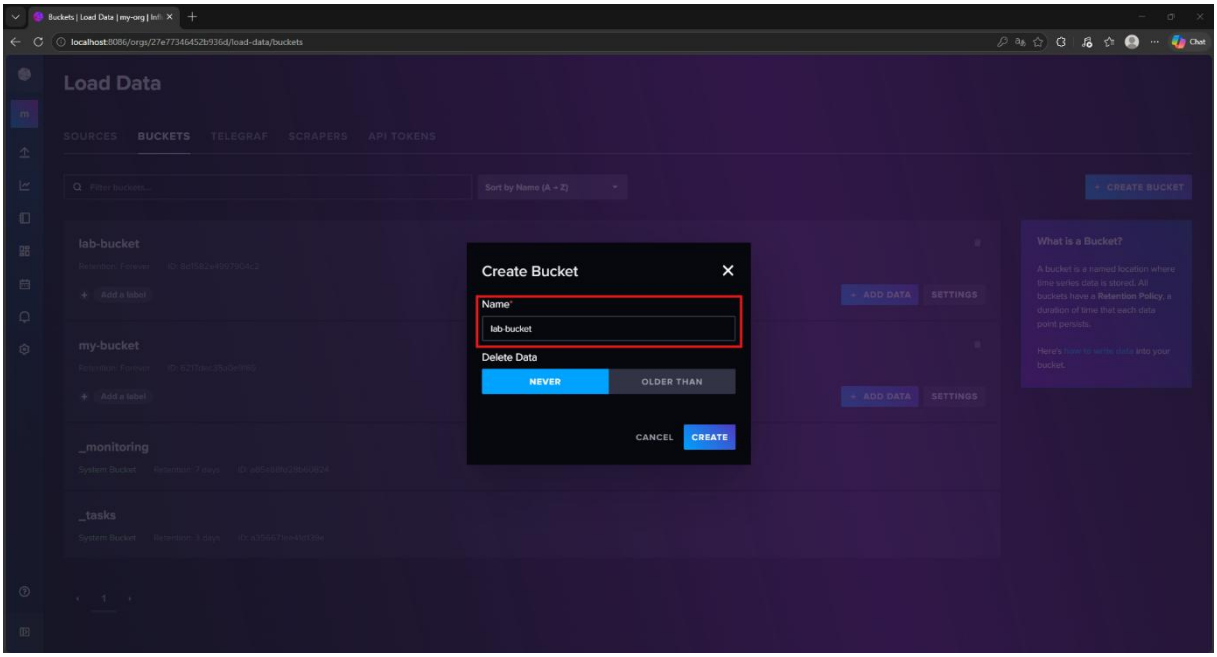
Navigiere im Hauptmenü auf der linken Seite zum Punkt **Buckets:**



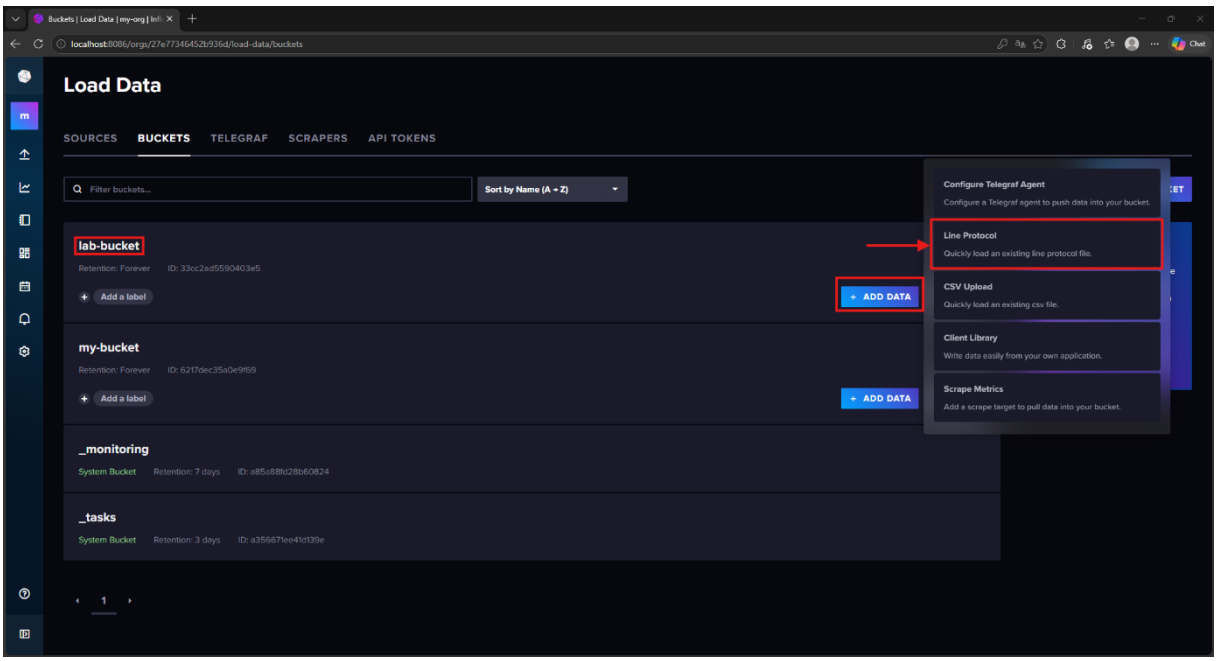
Erstelle nun einen neuen Bucket:



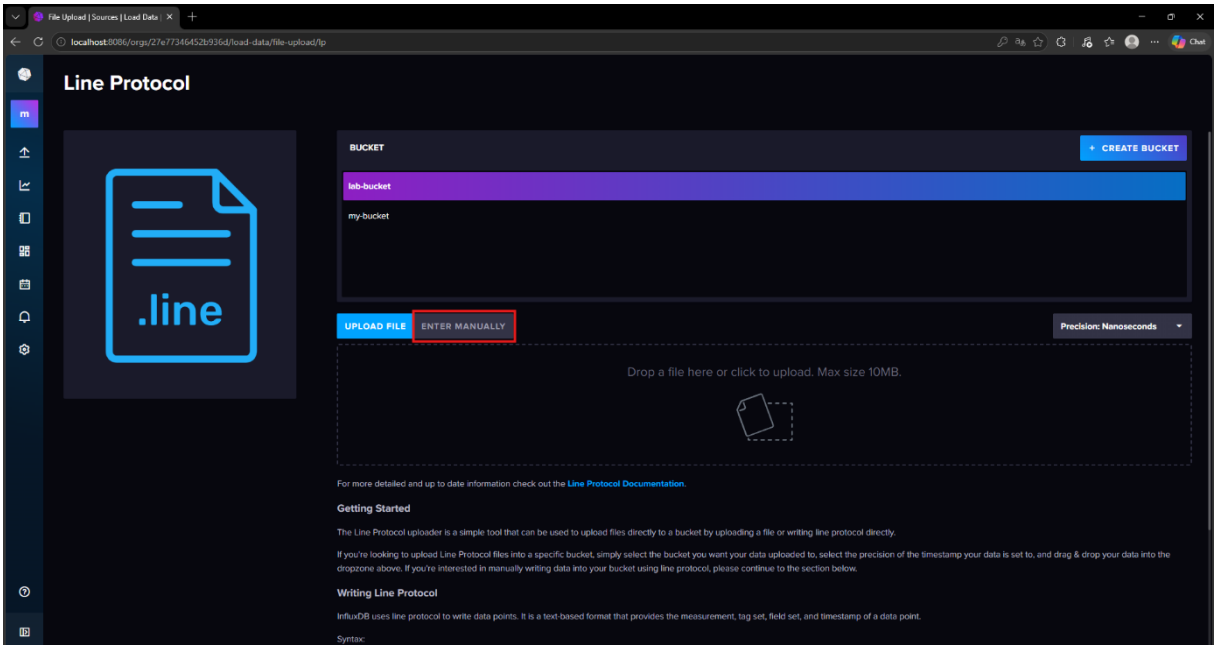
Benenne den Bucket **“lab-bucket“**:



Nachdem der Bucket in der Übersicht erscheint, lassen sich über **“ADD DATA“** die Testdaten importieren:

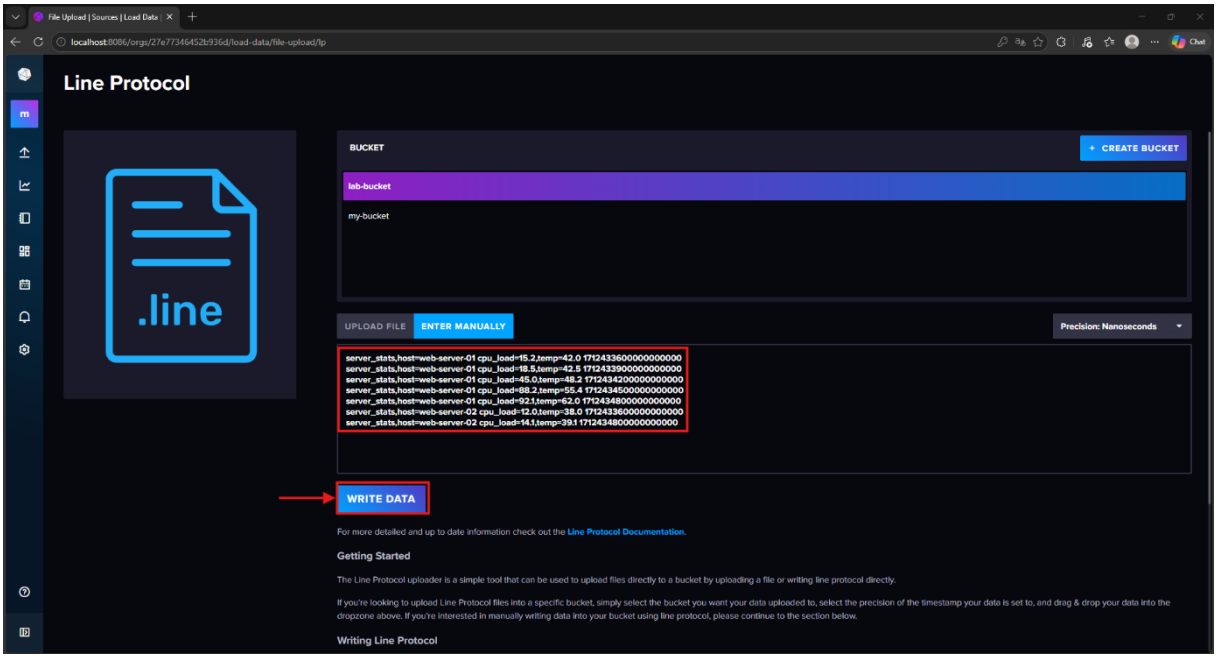


Nun wählen wir **“ENTER MANUALLY“** aus:



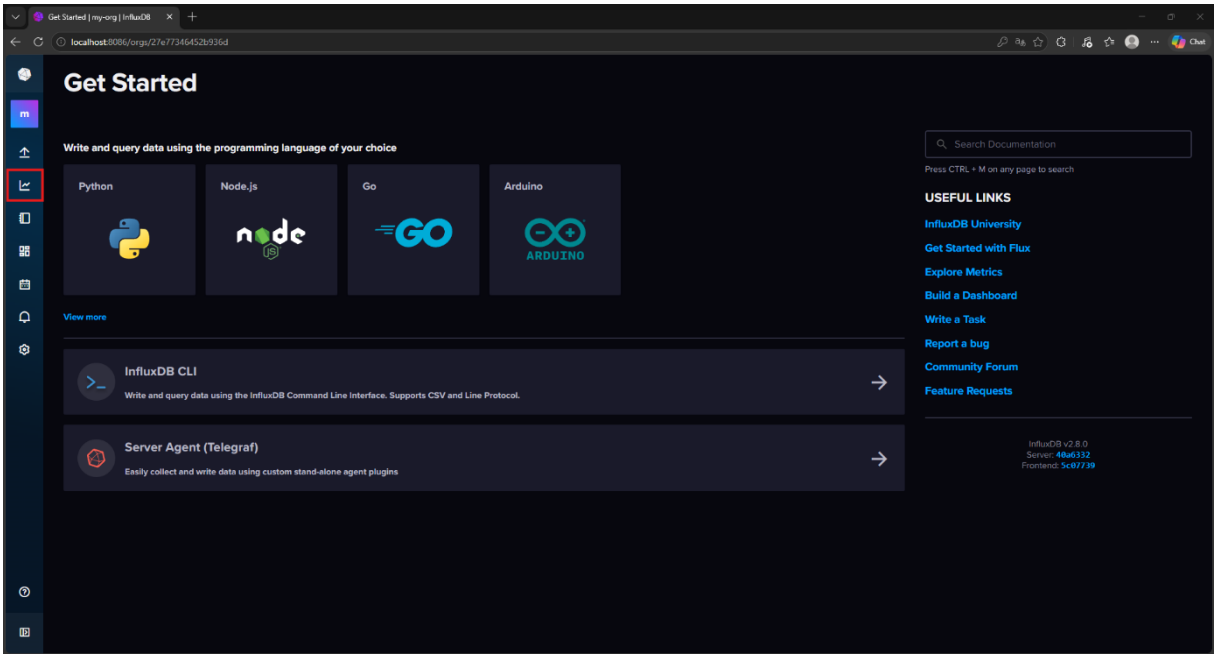
Testdatensatz:

```
server_stats,host=web-server-01 cpu_load=15.2,temp=42.0 1712433600000000000
server_stats,host=web-server-01 cpu_load=18.5,temp=42.5 1712433900000000000
server_stats,host=web-server-01 cpu_load=45.0,temp=48.2 1712434200000000000
server_stats,host=web-server-01 cpu_load=88.2,temp=55.4 1712434500000000000
server_stats,host=web-server-01 cpu_load=92.1,temp=62.0 1712434800000000000
server_stats,host=web-server-02 cpu_load=12.0,temp=38.0 1712433600000000000
server_stats,host=web-server-02 cpu_load=14.1,temp=39.1 1712434800000000000
```

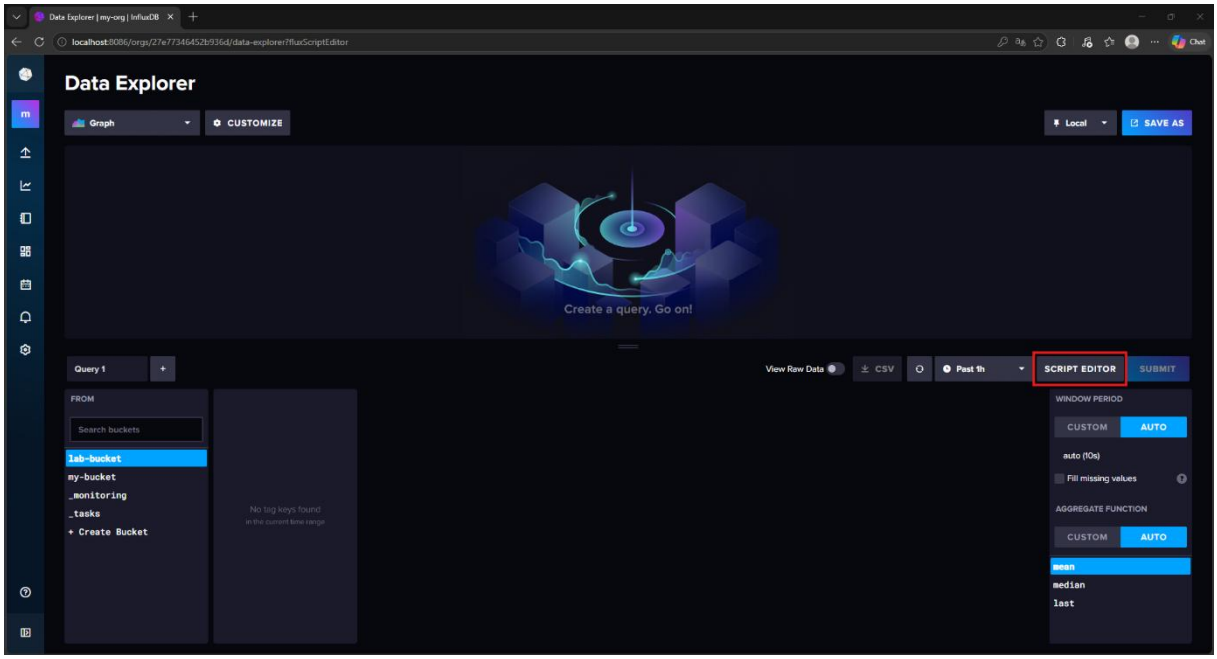


3. Daten abfragen (Flux Queries)

Navigiere im Hauptmenü auf der linken Seite zum **Graph Icon:**



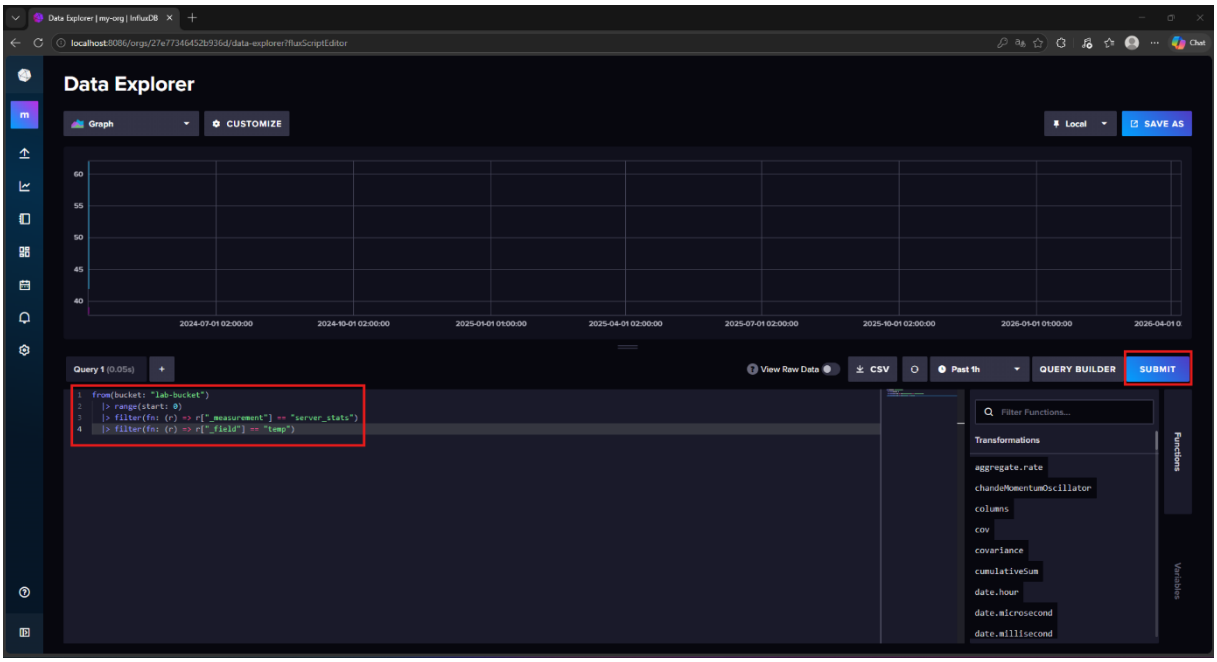
Nun öffnen wir den **“SCRIPT EDITOR“**:



Abfrage 1: Gesamter Temperaturverlauf

Zeigt alle Temperaturwerte aller Server an.

```
from(bucket: "lab-bucket")
  |> range(start: 0)
  |> filter(fn: (r) => r["_measurement"] == "server_stats")
  |> filter(fn: (r) => r["_field"] == "temp")
```



Nach der Bestätigung mit **"SUBMIT"** wird das Abfrageergebnis visualisiert. Über das Dropdown-Menü oberhalb des Graphen lassen sich zudem verschiedene **Darstellungsformen** und Anzeigeoptionen konfigurieren.

Abfrage 2: Kritische CPU-Last filtern

Zeigt nur Zeitpunkte, an denen die Last über 80% lag.

→ Die Vorgehensweise für Abfrage 2 ist **identisch** mit der von Abfrage 1!

```
from(bucket: "lab-bucket")
  |> range(start: 0)
  |> filter(fn: (r) => r["_field"] == "cpu_load")
  |> filter(fn: (r) => r["_value"] > 80.0)
```

4. Bereinigung (Cleanup)

Um das System nach dem Lab wieder "sauber" zu hinterlassen, müssen Container und Images entfernt werden.

- Container stoppen: Beendet die laufende Instanz.

  ```
  docker stop influxdb
  ```

- Container löschen: Entfernt die Container-Konfiguration.

  ```
  docker rm influxdb
  ```

- Image löschen: Entfernt das heruntergeladene Programm von der Festplatte.

  ```
  docker rmi influxdb:latest
  ```